

WORKSTATION

SUPER DUPER PRD — Engineering Intelligence & Project Operations Platform

Status: Master Product Requirements Document | Version: 1.0 | Date: 17 September 2026

North Star: WORKSTATION mengubah seluruh aktivitas engineering menjadi satu sumber informasi yang terukur, dapat ditelusuri, dan mudah dipahami oleh developer maupun manajemen.

0. Executive Summary

WORKSTATION adalah platform internal/enterprise untuk mengelola software engineering dari perencanaan sampai operasi: project management, work item, ticketing/ITSM, Git intelligence, code review, release, deployment, server/infrastructure, AI codebase intelligence, evidence, progress, health, reporting, knowledge base, dan audit.

Masalah utama yang diselesaikan adalah fragmentasi informasi. Aktivitas developer tersebar di ticket, GitHub, server, deployment, chat, dokumen, dan laporan manual. WORKSTATION menghubungkan sumber-sumber tersebut menjadi timeline dan metrik yang berbasis bukti.

Prinsip mutlak: AI membantu membaca dan menganalisis; sistem dan integrasi menyimpan fakta; manusia melakukan validasi pada keputusan yang membutuhkan judgement.

1. Product Vision & Objectives

- Single source of truth untuk aktivitas engineering.
- Progress dihitung dari evidence, bukan sekadar angka manual.
- Memberi developer ruang kerja operasional yang cepat.
- Memberi manager ringkasan delivery yang dapat dipahami tanpa harus membaca kode.
- Menghubungkan Ticket → Work Item → Branch → Commit → PR → Release → Deployment → Evidence.
- Memberikan observability atas dua lingkungan server utama: server lokal kantor dan Kontabo VPS.
- Mendeteksi risiko melalui AI tanpa menjadikan AI sebagai sumber kebenaran.
- Menyediakan laporan harian, mingguan, bulanan dan Project 360° yang traceable.
- Menjaga auditability, security, permission, dan riwayat perubahan.

2. Product Scope & Non-Goals

2.1 In Scope

- Multi-organization/project/team
- Project, module, milestone, sprint, work item
- Ticketing dan ITSM
- GitHub/GitLab/Bitbucket adapter
- Commit, branch, PR, review, release
- Deployment dan environment
- Server inventory, agent, health, services
- AI scan, finding, recommendation, technical debt
- Evidence & trust model
- Progress/health engine
- Incident, blocker, SLA
- Reporting & management summary
- Knowledge base & search

- Notification, integration, webhook
- RBAC, audit log, security

2.2 Non-Goals

- Menggantikan GitHub sebagai Git provider.
- Menganggap commit sebagai bukti otomatis durasi kerja.
- Membiarkan AI mengubah status bisnis tanpa kontrol.
- Menggunakan satu skor opaque sebagai penentu performa individu.
- Menyimpan secret/password server secara plaintext.
- Menjadikan persentase progress manual sebagai satu-satunya sumber laporan.

3. Target Users & Roles

Role	Fokus	Akses inti
Super Admin	Platform	Global configuration, security, audit, integrations
Organization Admin	Organisasi	Users, teams, projects, policies
Manager	Delivery	Dashboard, reports, health, progress, risks
Project Manager	Project	Planning, sprint, work items, tickets, reports
Tech Lead	Engineering	Architecture, PR, findings, deployment, quality
Developer	Execution	My Work, tickets, code, evidence, deployments
QA	Quality	Testing, tickets, release gates, evidence
Support	Service	Ticket intake, SLA, incidents, KB
Viewer	Read-only	Approved dashboards/reports

4. Organization & Project Model

Hierarchy: Organization → Teams/Users → Projects → Modules → Milestones → Sprints → Work Items/Tickets. Satu project dapat terhubung ke satu atau beberapa repository, environment, server, dan integration.

- Project metadata: owner, PM, tech lead, status, priority, target dates, repository links, environments.
- Project states: PLANNING, ACTIVE, ON_HOLD, AT_RISK, COMPLETED, ARCHIVED.
- Module dan milestone harus dapat memiliki target, owner, dependencies, acceptance criteria, evidence, dan progress.
- Project 360° menyatukan delivery, tickets, Git, deployments, infrastructure, AI, health, blockers, dan reports.

5. Work Item & Sprint Management

- Work item types: EPIC, FEATURE, TASK, SUBTASK, BUG, TECH_DEBT, SPIKE, MAINTENANCE.
- Fields: title, description, owner, priority, estimate, status, sprint, milestone, module, acceptance criteria, dependencies, due date.
- Status minimal: BACKLOG, READY, IN_PROGRESS, BLOCKED, IN_REVIEW, READY_FOR_TEST, TESTING, DONE, CANCELLED.
- Dependency graph wajib mendeteksi circular dependency.
- Sprint: goal, start/end, capacity, planned items, completed items, carry-over, blockers, retrospective notes.
- Progress sprint/project harus explainable dan dapat drill-down ke item serta evidence.

6. Ticketing / ITSM

6.1 Ticket Types

- BUG
- FEATURE_REQUEST

- TECHNICAL_ISSUE
- MAINTENANCE
- SECURITY
- PERFORMANCE
- QUESTION
- SERVICE_REQUEST
- INCIDENT

6.2 Ticket Workflow

NEW → TRIAGED → ASSIGNED → IN_PROGRESS → IN_REVIEW → READY_FOR_TEST → TESTING → READY_FOR_DEPLOY → DEPLOYED → RESOLVED → CLOSED. Exception states: BLOCKED, REOPENED, CANCELLED, DUPLICATE, WONT_FIX.

6.3 Ticket Data

- Ticket ID, requester, project, type, category, severity, priority, assignee, team, description, attachments, comments, SLA, linked work item, Git evidence, deployment, resolution, timestamps, history.
- Severity dan Priority dipisahkan: Severity menggambarkan dampak; Priority menggambarkan urutan penanganan.
- Duplicate detection oleh AI hanya berupa suggestion sampai dikonfirmasi manusia.

6.4 SLA & Incident

- Configurable first-response dan resolution target.
- SLA state: ON_TRACK, AT_RISK, BREACHED, PAUSED.
- Incident memiliki severity, affected service, commander/owner, timeline, impact, mitigation, root-cause notes, postmortem, corrective actions.
- Incident timeline harus immutable/auditable.

7. Git Intelligence

- Repository registry dengan provider, owner, URL, default branch, sync state.
- Branch tracking: name, type, protection, creator, latest commit, ahead/behind, linked work.
- Commit tracking: SHA, author, timestamp, message, files changed, additions/deletions, linked ticket/work item.
- PR tracking: title, author, source/target branch, status, reviewers, approvals, merge timestamp, linked work.
- Release tracking: tag/version, release notes, source commit, artifacts, deployment targets.
- Webhook-first synchronization; scheduled reconciliation menjadi fallback.
- Git activity adalah evidence aktivitas kode, bukan bukti tunggal jam kerja atau keberhasilan bisnis.

8. Deployment & Release Management

Entity	Minimum capability
Environment	DEV, STAGING, UAT, PRODUCTION; variables/policies/gates
Deployment	version/commit, branch, actor, environment, server, start/end, status, logs, rollback
Release	version, scope, notes, linked tickets, approvals, deployment state
Gate	test, approval, security/quality checks, required evidence
Rollback	target version, reason, actor, result, audit trail

Deployment record wajib dapat menjawab: apa yang dideploy, dari commit/release mana, oleh siapa/proses apa, kapan, ke environment/server mana, hasilnya apa, dan apa bukti keberhasilannya.

9. Server & Infrastructure

Platform harus mendukung minimal server lokal kantor dan Kontabo VPS melalui Workstation Agent.

- Server inventory: hostname, environment, OS, IP reference, region, owner, tags, status.

- Services: web server, PHP/runtime, database, queue worker, scheduler, Redis, Docker/container, custom services.
- Health telemetry: CPU, RAM, disk, network, uptime, process/service state, load, latency, deployment state.
- Agent mengirim heartbeat dan telemetry melalui koneksi aman; hindari SSH polling sebagai mekanisme utama.
- Threshold dan alert dapat dikonfigurasi per server/service/environment.
- Server credential/secret tidak boleh disimpan plaintext.

10. Workstation Agent

- Installable agent untuk Linux/server environment.
- Secure enrollment menggunakan one-time registration token atau certificate.
- Heartbeat, metrics, service discovery, deployment hooks, command/job execution terkontrol.
- Capability-based permissions: agent hanya boleh melakukan aksi yang diizinkan.
- Signed requests, replay protection, rotating credentials, audit trail.
- Offline buffer lalu retry ketika koneksi kembali.
- Agent version dan compatibility wajib tercatat.

11. AI Codebase Intelligence

11.1 AI Scan

- Full repository scan: architecture, code quality, security patterns, performance, testing, documentation, dependency/config risks, duplication, maintainability.
- Scan menghasilkan snapshot dan findings; hasil tidak menimpa source code atau status bisnis secara otomatis.

11.2 AI Finding

- Category, severity, confidence, evidence location, explanation, impact, suggested remediation, detected_at, scanner version, status.
- Finding states: PENDING, CONFIRMED, REJECTED, FALSE_POSITIVE, IN_PROGRESS, RESOLVED, ACCEPTED_RISK.

11.3 AI Recommendation

- Prioritized recommendations dengan alasan, expected impact, effort estimate, affected module, evidence, confidence.
- Recommendation dapat diubah menjadi work item setelah approval manusia.

11.4 Technical Debt

- Debt item memiliki origin, evidence, impact, effort, owner, target milestone, status, dan aging.
- AI tidak boleh menciptakan debt sebagai fakta tanpa evidence.

12. Evidence & Data Trust Model

Setiap data analitis penting harus memiliki metadata: Source, Source ID, Timestamp, Confidence, Verification Status, Last Updated, Actor/System, dan Relationship.

Evidence	Contoh
Git	Commit SHA, PR, review
Work Management	Ticket, task, acceptance criteria
Quality	Test run, QA result
Deployment	Deployment ID, version, environment
Infrastructure	Agent telemetry, service status
AI	Scan ID, finding evidence, scanner version
Human	Comment, approval, verification

Verification states: UNVERIFIED, SYSTEM_VERIFIED, HUMAN_VERIFIED, DISPUTED, REJECTED. UI wajib membedakan fakta, inference, recommendation, dan manual input.

13. Engineering Event & Timeline

Event bus/domain event menjadi fondasi audit dan analytics.

- CODE_COMMITTED
- BRANCH_CREATED
- PR_CREATED
- PR_APPROVED
- PR_MERGED
- TASK_CREATED
- TASK_COMPLETED
- TICKET_CREATED
- TICKET_RESOLVED
- AI_SCAN_COMPLETED
- FINDING_CREATED
- DEPLOYMENT_STARTED
- DEPLOYMENT_SUCCESS
- DEPLOYMENT_FAILED
- SERVER_DOWN
- SERVER_RECOVERED
- RELEASE_CREATED
- SPRINT_STARTED
- SPRINT_COMPLETED

Setiap event memiliki event_id, type, actor/source, entity, timestamp, payload schema version, correlation_id, dan audit metadata.

14. Progress Engine

Progress adalah output terhitung. Sistem harus selalu dapat menjelaskan dari item mana angka tersebut berasal.

- Level 1: task/work item completion.
- Level 2: milestone/feature completion berbasis child items dan acceptance criteria.
- Level 3: project progress berbasis weighted deliverables.
- Optional evidence gates: review, test, deployment, acceptance.
- Manual override diperbolehkan hanya dengan alasan, actor, timestamp, dan audit trail.
- Contoh: Progress = weighted completion deliverables; completion tidak otomatis berarti deployed atau accepted.

15. Project Health Engine

Health berbeda dari progress. Project dapat 80% progress tetapi memiliki risiko tinggi.

Dimension	Input contoh
Delivery	schedule variance, overdue, blockers
Quality	bugs, reopen rate, test coverage/result
Security	open findings, severity, remediation age
Performance	latency/error indicators
Testing	test pass/fail, critical coverage
Documentation	missing/aging documentation
Infrastructure	server health, failed deployments

Health output harus explainable: dimension, evidence, threshold, trend, timestamp. Hindari opaque scoring.

16. Blocker & Risk Management

- Blocker: description, owner, impact, dependency, created_at, due date, resolution, linked evidence.

- Risk: probability, impact, exposure, mitigation, contingency, owner, review date.
- Risk register tampil pada project dashboard dan report.
- AI dapat mengusulkan risk berdasarkan evidence tetapi manusia mengonfirmasi.

17. Reporting & Management Translation

17.1 Daily Report

- Completed work, active work, blockers, tickets, commits/PRs, deployments, incidents, server issues, AI findings, next actions.

17.2 Weekly Report

- Sprint outcome, planned vs completed, delivery trend, ticket/SLA trend, Git activity, releases/deployments, quality/security, infrastructure, risks, next week plan.

17.3 Monthly Report

- Project portfolio, milestone progress, release history, incident/SLA, quality/security trend, technical debt, infrastructure trend, major accomplishments and risks.

17.4 Dual Language

Satu evidence source menghasilkan dua presentation layers: Engineering View untuk developer/tech lead dan Management View untuk manager. Contoh: teknis 'PR merged after fixing N+1 query' dapat diterjemahkan menjadi 'perbaikan efisiensi akses database pada modul X' tanpa menghilangkan link evidence.

18. Dashboard & UX

- Global Dashboard: portfolio, delivery, health, tickets, deployments, infrastructure, risks.
- My Work: assigned tickets/work items, review queue, blockers, recent activity, deployment tasks.
- Project 360°: Overview, Roadmap, Sprint, Work, Tickets, Git, Releases, Deployments, Servers, AI, Quality, Risks, Reports, Knowledge.
- What Happened?: chronological daily summary with evidence links.
- Ticket workspace: fast intake, keyboard-friendly triage, filters, saved views, bulk actions, SLA visibility.
- Every important metric must support drill-down to source evidence.

19. Knowledge Base & Search

- Resolved ticket dapat menjadi draft knowledge article.
- Article versioning, owner, review date, tags, related projects/tickets/incidents.
- Global search across tickets, work items, Git metadata, releases, findings, servers, reports, KB.
- Search results must indicate entity type, project, timestamp, and relevance context.

20. Notification Center

- In-app notifications; email/chat adapter optional.
- Events: assignment, mention, SLA risk/breach, deployment failure, server outage, critical AI finding, PR review request, blocker, report ready.
- Notification preferences per user/team/project.
- Deduplication dan rate limiting untuk mencegah notification storm.

21. Integration & Webhook Layer

- Provider adapters: GitHub/GitLab/Bitbucket; CI/CD; email/chat; monitoring where required.

- Inbound webhook validation: signature, timestamp, replay protection.
- Outbound webhooks: configurable event filters, retries, dead-letter state, delivery logs.
- Idempotency key wajib pada event ingestion dan deployment callbacks.
- Integration secrets encrypted at rest and masked in UI/logs.

22. Data Architecture

Recommended logical stack: relational database as system of record; queue/event processing for integrations and telemetry; object storage for attachments/artifacts; search index for global search; cache for read-heavy dashboard paths. Exact technologies dapat dipilih saat technical design tanpa mengubah domain model.

22.1 Core Entities

- organizations, users, teams, roles, permissions, team_members, organization_members
- projects, project_members, project_modules, milestones
- sprints, work_items, work_item_dependencies
- tickets, ticket_comments, ticket_attachments, ticket_history, ticket_sla
- repositories, branches, commits, pull_requests, pull_request_reviews, releases
- environments, servers, server_services, deployments, deployment_logs
- ai_scans, ai_findings, ai_recommendations, technical_debts
- activities, time_entries, engineering_events
- incidents, incident_timelines, postmortems
- knowledge_articles, knowledge_versions
- reports, report_sections, notifications, audit_logs
- integrations, webhooks, webhook_deliveries
- evidence, evidence_links, risks, blockers

22.2 Core Relationships

Organization 1..N Projects/Users/Teams. Project 1..N Modules/Milestones/Sprints/Tickets/Work Items/Repositories/Environments/Servers/AI Scans/Reports. Work Item N..N Evidence and Dependencies. Repository 1..N Branches/Commits/PRs/Releases. Release 1..N Deployments. Environment 1..N Servers. AI Scan 1..N Findings/Recommendations. Ticket dapat terhubung ke Work Item, Commit, PR, Release, Deployment, Incident, dan KB.

23. API & Service Boundaries

- Auth & Identity Service
- Organization/Project Service
- Work Management Service
- Ticketing Service
- Git Integration Service
- Release/Deployment Service
- Infrastructure/Agent Service
- AI Intelligence Service
- Evidence/Analytics Service
- Reporting Service
- Notification Service
- Knowledge/Search Service
- Audit/Security Service

API principles: REST/JSON for external/admin workflows, webhook endpoints for integrations, async jobs for long-running scans/syncs, pagination/filtering/sorting, idempotency, versioning, authorization at resource level, structured errors, correlation IDs.

24. Security Architecture

- RBAC + granular permission policies.
- 2FA for privileged roles; session/device controls.
- Encryption in transit and at rest.
- Secrets manager/encrypted credential store; never plaintext.
- Webhook signature validation and replay protection.
- Audit logs for permission, configuration, deployment, report, and verification changes.
- Rate limiting, brute-force protection, CSRF protection where applicable, secure headers.
- Least privilege for server agent and integrations.
- PII minimization and configurable retention.
- Backup, restore testing, disaster recovery, incident response.

25. Auditability & Governance

Audit log is append-oriented and records actor, action, target, before/after where appropriate, timestamp, source, request/correlation ID, and reason for sensitive overrides. Deletion of important evidence should be soft-delete or governed retention, with elevated permissions and audit trail.

26. Accuracy & Data Quality Rules

1. Never infer a business outcome solely from commit count.
2. Never infer work duration solely from Git timestamps.
3. Never treat AI confidence as factual certainty.
4. Every computed KPI must expose formula/version and source records.
5. Conflicting source data must be marked DISPUTED rather than silently overwritten.
6. Webhook ingestion must be idempotent.
7. Time is stored consistently and displayed in user/project timezone.
8. Manual overrides require reason and audit trail.
9. Reports are generated from frozen/report snapshots so historical reports remain reproducible.

27. End-to-End Reference Flow

A user reports a printing bug → Ticket NEW → AI suggests category/duplicates → human TRIAGES → Work Item created → developer assigned → branch linked → commits reference ticket → PR opened/reviewed/merged → tests pass → Release created → deployment to Staging → QA evidence → Production approval → deployment success → Ticket RESOLVED → KB draft created → daily/weekly report aggregates the evidence. Project progress and health update from the resulting state, not from a manually invented percentage.

28. Reporting Metrics

Area	Metrics
Delivery	Progress, milestone completion, overdue, cycle time, lead time
Work	WIP, throughput, blocked time, sprint completion
Tickets	Open, resolved, reopen, severity, SLA compliance, resolution time
Git	Commits, PR cycle time, review time, merge rate, branch age
Release	Release frequency, deployment frequency, failure/rollback rate
Quality	Defect trend, test outcomes, escaped defects
Security	Open findings, severity, aging, remediation
Infrastructure	Uptime, incidents, service health, disk/CPU/RAM trend

AI	Scans, findings by status/severity/confidence, remediation rate
Knowledge	Articles, freshness, ticket-to-KB conversion

29. Acceptance Criteria — Platform

10. All major entities have stable IDs and audit timestamps.
11. Every dashboard metric can drill down to underlying records.
12. Ticket lifecycle supports all required states and history.
13. Git sync handles webhook delivery, retries, and reconciliation.
14. Deployment records link to release/commit/environment/server.
15. Server agent heartbeat and health data are securely ingested.
16. AI findings always store evidence and scanner metadata.
17. Progress calculation is reproducible and explainable.
18. Management report uses readable language while preserving evidence links.
19. RBAC prevents unauthorized project/server/deployment access.
20. Sensitive configuration changes generate audit events.
21. Historical reports remain reproducible from report snapshots.

30. MVP → V1 → V2 → Enterprise

Phase	Scope
MVP	Auth/RBAC, organization/project, work items, tickets, GitHub sync, basic releases/deployments, dashboard, audit log
V1	Sprints/milestones, SLA/incident, server agent, health monitoring, evidence, progress/health engine, daily/weekly/monthly reports, KB/search
V2	AI codebase scanner, AI findings/recommendations, technical debt, advanced analytics, multi-provider Git, CI/CD adapters, advanced deployment gates
Enterprise	Multi-org governance, SSO/SCIM, advanced policy engine, HA, DR, data retention policies, enterprise observability, advanced audit/compliance

31. Recommended Build Order

22. Foundation: identity, RBAC, organization, audit, core database.
23. Project/work management: project, module, milestone, sprint, work item.
24. Ticketing: intake, workflow, comments, attachments, SLA.
25. Git intelligence: repositories, branches, commits, PRs, webhook/reconciliation.
26. Release/deployment: environments, releases, deployment records, gates.
27. Server agent: enrollment, heartbeat, telemetry, services.
28. Evidence/event layer: correlation and traceability.
29. Progress/health analytics and Project 360°.
30. Reporting and management translation.
31. AI scanner/findings/recommendations and technical debt.
32. Knowledge, advanced search, integrations, enterprise hardening.

32. Definition of Done

- Feature has acceptance criteria.
- Relevant tests exist and pass.
- Required review is complete.
- Evidence links are attached.

- Deployment state is known when applicable.
- Documentation/KB updated when required.
- Security impact reviewed for sensitive changes.
- Audit trail exists.
- Status reflects actual state rather than intended state.

33. Product Principles — Final

- Evidence over assumption.
- Traceability over vanity metrics.
- AI as analyst, not authority.
- Automation with human control.
- Technical truth translated into business clarity.
- One platform, one timeline, multiple views.
- Every important number must be explainable.
- Security and audit are built-in, not bolted-on.
- Complexity must remain logical and navigable.
- Fast for developers; clear for managers.

34. Final Product Definition

WORKSTATION bukan sekadar task manager, ticketing system, Git dashboard, deployment panel, server monitor, atau AI code scanner. WORKSTATION adalah lapisan intelligence dan operations yang menghubungkan semuanya. Tujuan akhirnya adalah membuat organisasi dapat menjawab secara konsisten: apa yang dikerjakan, mengapa dikerjakan, siapa yang mengerjakan, bukti apa yang tersedia, apa yang berubah di code, apakah sudah direview dan diuji, kapan dan ke mana dideploy, bagaimana kondisi server, apa risikonya, dan apa langkah berikutnya.

Dokumen ini menjadi Master PRD. Technical design, ERD final, API contract, UI specification, architecture decision records, test plan, dan implementation backlog harus diturunkan dari prinsip serta requirements dalam dokumen ini tanpa menghilangkan traceability.

Appendix A — Suggested Navigation

- Overview
- My Work
- Projects
- Tickets
- Sprints
- Roadmap
- Git
- Releases
- Deployments
- Infrastructure
- AI Intelligence
- Quality & Security
- Incidents
- Reports
- Knowledge Base
- Search
- Notifications
- Administration
- Audit Logs

Appendix B — Core Status Vocabulary

Status names should remain consistent across API, database, UI, analytics, and reports. Display labels may be translated to Indonesian while preserving stable machine values.

Appendix C — Example Manager Summary

Project X menunjukkan kemajuan deliverable berdasarkan item yang telah selesai dan acceptance criteria yang terpenuhi. Dalam periode laporan, beberapa work item selesai, sejumlah ticket terselesaikan, perubahan kode telah melalui review, dan release tertentu berhasil dideploy ke environment yang dituju. Risiko yang masih terbuka, blocker, hasil pengujian, temuan keamanan, serta kondisi infrastructure ditampilkan bersama evidence agar manager dapat melihat konteks di balik angka.

END OF MASTER PRD